

Full Fine-Tuning vs. LoRA: Performance, Efficiency, and Adaptation Analysis

Jiwoo Park

1. 개요

Full Fine-Tuning 방식과 LoRA Fine-Tuning 방식 비교를 위한 실험 진행.

Full Fine-Tuning 에 비해 LoRA Fine-Tuning 가지는 더 적은 메모리, 더 적은 저장용량, 계산량 등을 재현 및 검증을 목표.

이번 실험의 목표는 다음과 같음.

RQ1. Full Fine-Tuning 과 LoRA 의 Performance–Efficiency Trade-off 는 어떻게 되는가?

RQ2. LoRA Configuration 에 따른 Performance 와 Parameter Efficiency 차이는 어떻게 되는가?

RQ3. LoRA 의 Task Adaptation 은 어느 Layer 에 집중되며, 선택적 Layer 학습으로 성능을 유지할 수 있는가? (*Additional*)

Statistic	PQA-L	PQA-U	PQA-A
Number of QA Pairs	1.0k	61.2k	211.3k
Yes (%)	55.2	–	92.8
No (%)	33.8	–	7.2
Maybe (%)	11.0	–	0.0
Average Question Length	14.4	15.0	16.3
Average Context Length	238.9	237.3	238.0
Average Long Answer Length	43.2	45.9	41.0

Table 1: Statistics of the PQA-L, PQA-U, and PQA-A subsets in the PubMedQA dataset.

2. 데이터 설명 및 전처리

[PubMedQA](#)에서 제시한 Dataset 을 활용하여 Label Classification QA 로 사용. Abstract context 와 이에 대응되는 biomedical research question 를 바탕으로, 질문에 대한 결론을 **3-class label**(yes, no, maybe) 중 하나로 답하는 형식의 task 로 구성됨..

[PubMedQA](#) 는 인공지능 annotation 기반 PQA-A, 라벨이 존재하지 않는 PQA-U, 전문가 annotation 기반 PQA-L subset 으로 구성되어 있음. 상세 구성 [Table 1](#) 참고

학습 시 모델의 Class 편향 방지를 위해 [Table 2](#) 과 같은 구성으로 Dataset Seed Based 랜덤 샘플링. (Seed: 42)

Dataset	Yes Label	No Label	Maybe Label	Source
Train	5k (50%)	5k (50%)	-	PQA-A
Validate	1k (50%)	1k (50%)	-	PQA-A
Test	276 (55.2%)	169 (33.8%)	55 (11%)	PQA-L

Table2: Statistics of the Experimental Dataset Splits

3. 방법론 및 코드 구현 설명

3.1 전체 학습 구조

전처리된 PubMedQA Dataset 을 동일한 Prompt Template 으로 변환하여 Full Fine-Tuning 과 LoRA Fine-Tuning 을 수행함. 모든 방법에서 동일한 입출력 형식과 Training Objective 를 사용하며, 학습 완료 후 동일한 Evaluation Pipeline 을 적용함.

3.2 Prompt 구성 및 출력 형식

Zero-shot Prompt 기반 Question 과 Abstract Context 만을 근거로 답변을 판단하도록 구성하여, 출력은 yes, no, maybe 중 하나로 제한함.

사용된 프롬프트는 [Appendix A1](#) 에 작성되어 있음.

3.3 Training Objective

Full Fine-Tuning 과 LoRA 모두 동일한 **Causal Language Modeling Objective** 를 사용함. Prompt Token 은 Label Masking 을 통해 Loss 계산에서 제외하고, Assistant 의 Ground-truth Answer Token 에 대해서만 Loss 를 계산함.

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log P_{\theta} (y_t | x, y_{<t})$$

여기서 x 는 Instruction Prompt, y 는 Ground-truth Answer 를 의미함.

3.4 Full Fine-Tuning

Pre-trained Model 의 전체 Parameter 를 Trainable 상태로 설정하고, Training Loss 에 대한 Gradient 를 통해 전체 Weight 를 Update 함.

이를 통해 Pre-trained Model 의 전체 Parameter Space 를 활용하여 PubMedQA Task 에 Adaptation 하도록 구성함.

3.5 LoRA Fine-Tuning

Pre-trained Weight W_0 를 Frozen 상태로 유지하고, Target Linear Layer 에 Low-rank Matrix A, B 를 추가하여 Task-specific Update 를 학습함.

$$W' = W_0 + \Delta W \qquad \Delta W = \frac{\alpha}{r} BA$$

여기서 r 은 Rank, α 는 Scaling Factor 를 의미함. Target Module, Rank 및 Target Layer 를 변경할 수 있도록 구현하여 서로 다른 LoRA Configuration 을 동일한 학습 구조에서 적용함.

3.6 Layer-wise Update 분석 및 Selective LoRA (Additional)

All-layer LoRA 의 Checkpoint 에서 각 Transformer Layer 의 Effective Update 를 복원하여 Parameter Update 가 집중되는 Layer 를 분석함.

Layer 별 Update Magnitude 를 기준으로 High-update Layer 와 Low-update Layer 를 선정함. 이후 선택된 Layer 에만 LoRA 를 적용하는 Selective LoRA 를 수행하여, Update 가 집중된 Layer 만을 학습 대상으로 제한했을 때의 Task Adaptation 성능을 비교함.

4. 실험설정

4.1 공통 학습 환경

모든 실험은 Qwen3-1.7B 를 Base Model 로 사용하고, NVIDIA RTX 3090 1 장의 Single-GPU 환경에서 수행함. BF16 Precision 과 SDPA Attention Backend 를 사용하며, 실험의 Reproducibility 를 위해 Random Seed 를 42 로 고정함.

Full Fine-Tuning 과 LoRA 는 동일한 Dataset, Prompt Template, Batch 구성, Optimizer 및 Evaluation Procedure 를 사용함.

자세한 실험 조건은 [Appendix A2](#) 참고

4.2 평가 지표

Performance

모델의 Task Performance 는 다음 지표로 평가함.

- Accuracy
- Macro-F1
- Class-wise F1
- Confusion Matrix

Validation Set 은 yes/no 로만 구성되어 있으므로 **Validation Macro-F1 은 2-class 기준으로** 계산함. 최종 PQA-L Test 에서는 yes/no/maybe 전체 Label 을 사용하여 **3-class 기준으로** 평가함.

Efficiency

Full Fine-Tuning 과 LoRA 의 Efficiency 비교를 위해 다음 항목을 측정함.

- Total / Trainable Parameters 및 Trainable Ratio
- Training Time 및 sec/step
- Peak Allocated / Reserved GPU Memory
- Checkpoint Size
- Sample / Token Throughput
- Inference Latency / Throughput

4.3 Selective LoRA

RQ3 에서는 Layer-wise LoRA Update Magnitude 를 기준으로 Transformer Layer 를 Ranking 함.

상위 4 개 Layer 를 High-update Layer, 하위 4 개 Layer 를 Low-update Layer 로 선정함.

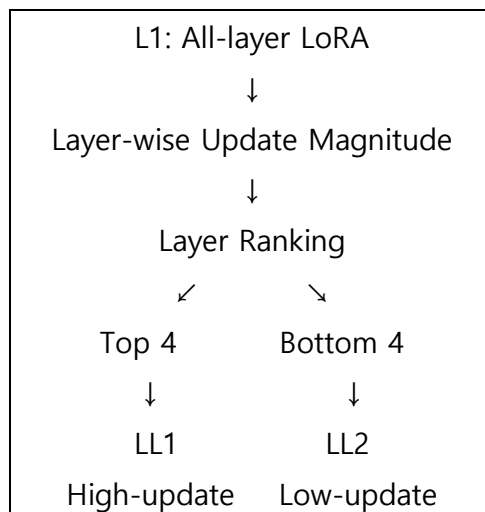


Figure 1. Selective LoRA layer selection procedure based on layer-wise update magnitude.

LL1 과 LL2 에는 선택된 Layer 의 q_proj , v_proj 에만 LoRA 를 적용함. 이를 All-layer LoRA 인 L1 과 비교하여 선택적 Layer 학습에 따른 Performance 유지 여부와 Trainable Parameter 감소 효과를 분석함.

5. 실험결과 및 분석

Base Model Selection

본 연구에서는 decoder-only Dense Language Model 을 대상으로 Full Fine-Tuning 과 LoRA Fine-Tuning 을 비교함. Qwen3-1.7B 는 (1) Mixture-of-Experts(MoE)가 아닌 Dense Transformer 구조로 구성되어 있어 Parameter Update 방식에 따른 차이를 비교하기 용이하고, (2) Chat Template 기반의 Causal Language Model 로 PubMedQA 의 Instruction-based QA 형식을 적용할 수 있으며, (3) 단일 NVIDIA RTX 3090 GPU 환경에서 BF16 기반 Full Fine-Tuning 이 가능한 모델 규모임. 이러한 조건을 바탕으로 Qwen3-1.7B 를 Base Model 로 선정함. 이를 통해 모델 Architecture 의 차이가 아닌 Full-parameter Update 와 Low-rank Adapter Update 방식의 차이에 초점을 두고 비교함. 자세한 비교표는 [Appendix A3](#) 참고.

RQ1. Full Fine-Tuning 과 LoRA 의 Performance–Efficiency Trade-off 는 어떻게 되는가?

Training 및 Validation Loss

Figure 2 에서 Full Fine-Tuning 과 LoRA 모두 학습 초기에 Training Loss 가 빠르게 감소한 이후 약 10^{-1} 수준에서 안정적으로 수렴하는 양상을 보임. 두 방법의 Training Loss Curve 가 전반적으로 유사하게 형성되어, LoRA 의 제한된 Trainable Parameters 만으로도 Full Fine-Tuning 과 유사한 수준으로 Training Objective 를 최적화할 수 있음을 확인함.

Validation Loss 역시 학습 진행에 따라 지속적으로 감소함. 특히 25% Checkpoint 이후 감소 폭이 크게 줄어들며, 학습 후반부에는 두 방법 모두 유사한 수준으로 수렴함. Full Fine-Tuning 과 LoRA 사이에서 뚜렷한 Validation Loss 차이가 나타나지 않아, LoRA 의 Parameter 제약이 본 실험에서 Validation 성능 저하로 직접 이어지지는 않은 것으로 확인됨.

따라서 1 Epoch 범위에서는 두 방법 모두 안정적인 Convergence 를 보였으며, Loss 관점에서 Full Fine-Tuning 의 추가적인 Adaptation Capacity 가 뚜렷한 이점으로 나타나지 않음.

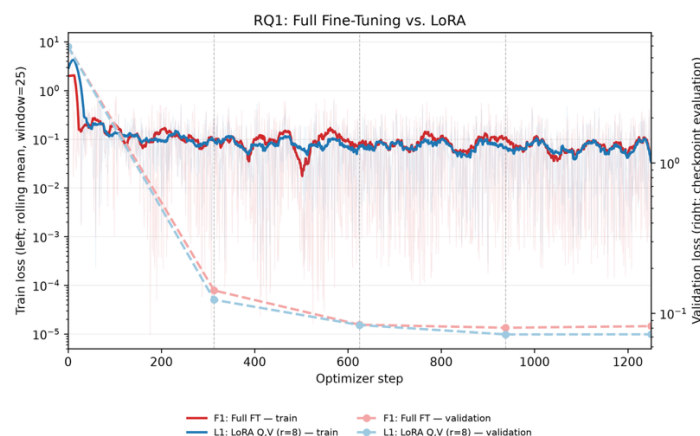


Figure 2. Training and Validation Loss of Full Fine-Tuning and LoRA

Test Performance

Baseline 대비 두 Fine-Tuning 방법 모두 Accuracy 가 0.114 point, Macro-F1 이 0.097 point 증가하여 PubMedQA 에 대한 Task Adaptation 효과가 명확하게 나타남. 반면 Full Fine-Tuning 과 LoRA 사이에서는 전체 성능 차이가 관찰되지 않음. Figure 3 참고

Class-wise F1 에서도 유사한 결과가 나타남. yes class 는 Full Fine-Tuning 과 LoRA 모두 0.801 로 동일하였으며, no class 에서는 각각 0.724 와 0.727 로 LoRA 가 0.003 높은 수준의 미세한 차이만 나타남.

다만 maybe class 에서는 Full Fine-Tuning 과 LoRA 모두 $F1 = 0$ 을 기록함. 이는 Fine-Tuning 에 사용한 PQA-A Train/Validation Dataset 이 yes/no 만으로 구성되어 maybe class 에 대한 supervision 이 제공되지 않은 데이터 구성과 관련된 것으로 판단됨. 특히 Fine-Tuning 이후 Baseline 에서 일부 존재하던 maybe class 의 예측 성능이 소실된 점에서, 학습 Dataset 의 Label Space 가 모델의 출력 분포에 영향을 미친 것으로 해석할 수 있음. 따라서 전체 Macro-F1 이 Accuracy 에 비해 상대적으로 낮게 나타난 주요 원인 역시 maybe class 의 성능 저하와 관련됨.

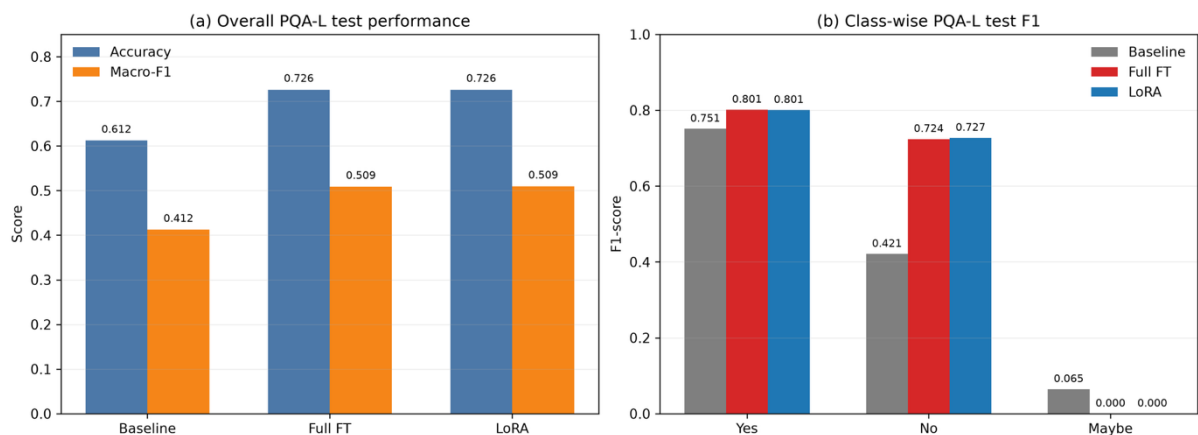


Figure 3. PQA-L test performance comparison of baseline, Full Fine-Tuning, and LoRA.

Training 및 Resource Efficiency

동일한 Test Performance 를 달성했음에도 Resource Efficiency 에서는 LoRA 의 뚜렷한 이점이 확인됨. [Table 3](#) 과 같이 LoRA 는 Full Fine-Tuning 대비 Trainable Parameters 와 Checkpoint Size 를 크게 줄였으며, Training Time 또한 감소함. 이를 통해 전체 Parameter 를 Update 하지 않고도 제한된 Low-rank Parameter 만으로 유사한 Task Adaptation 이 가능했음을 확인함.

반면 Training Peak VRAM 은 LoRA 에서 소폭 높게 측정됨. 이는 Full Fine-Tuning 에만 적용된 Gradient Checkpointing 으로 인한 Activation Memory 절감 효과의 영향을 받은 것으로 판단됨. Inference VRAM 은 LoRA 가 낮게 측정되었으나, 동일한 Base Model 을 사용하는 만큼 Loading 및 Memory Management 방식의 영향이 포함될 수 있어 LoRA 자체의 구조적 이점으로 단정하지 않음.

종합적으로 LoRA 는 동일한 Task Performance 를 유지하면서 Parameter, Training Time 및 Storage 측면에서 높은 Efficiency 를 확보함. 다만 Memory Efficiency 는 실행 조건의 영향을 함께 받으므로 직접적인 비교에는 제한이 있음.

지표	Full FT	LoRA
Trainable Parameters	1.72B	1.61M (↓ 99.91%)
Training Time	3451.7 s	2488.3 s (↓ 27.91%)
Best Checkpoint Size	3.46 GB	22.3 MB (↓ 99.36%)
Peak Allocated VRAM	7.94 GB	8.45 GB (↑ 6.42%)
Inference Allocated VRAM	19.64 GB	13.96 GB (↓ 28.92%)

Table 3. Training and Resource Efficiency of Full Fine-Tuning and LoRA

RQ2. LoRA Configuration 에 따른 Performance 와 Parameter Efficiency 차이는 어떻게 되는가?

LoRA 의 Target Module 과 Rank 에 따른 영향을 확인하기 위해 서로 다른 LoRA Configuration 의 학습 과정과 Test Performance, Parameter Efficiency 를 비교함.

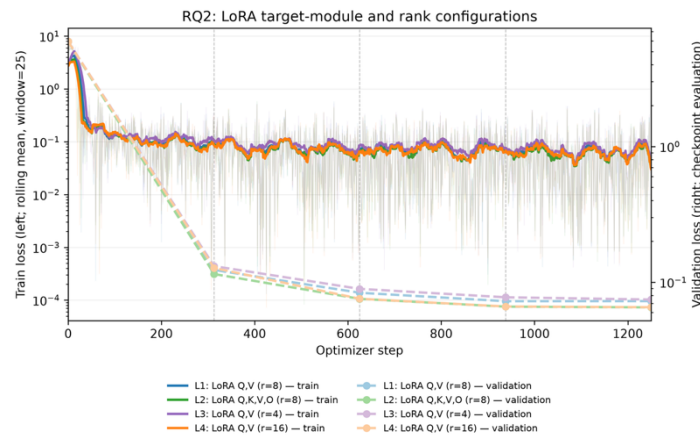


Figure 4. Training and Validation Loss across LoRA Target Module and Rank Configurations

모든 LoRA Configuration 에서 유사한 Training 및 Validation Loss 수렴 양상이 나타남. Configuration 에 따른 뚜렷한 학습 불안정성은 관찰되지 않았으며, 이후 Test Performance 차이는 특정 Configuration 의 학습 실패보다는 Rank 와 Target Module 설정에 따른 차이로 분석함.

Run	Target Module	Rank	Trainable Parameters	Test Acc.	Test Macro-F1	Training Time	Checkpoint Size
L3	Q, V	4	0.80M (0.0466%)	0.714	0.501	2,497 s	19.1 MB
L1	Q, V	8	1.61M (0.0932%)	0.726	0.509	2,488 s	22.3 MB
L4	Q, V	16	3.21M (0.1863%)	0.730	0.512	2,496 s	28.8 MB
L2	Q, K, V, O	8	3.21M (0.1863%)	0.720	0.505	2,705 s	28.8 MB

Table 4. LoRA Configuration–Performance–Efficiency Summary
(Trainable parameter ratio is reported relative to the total parameters of base model)

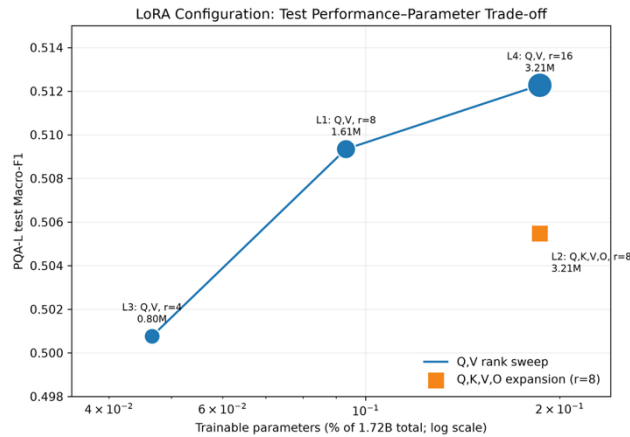


Figure 5. LoRA Configuration: Test Performance-Parameter Trade-off

Figure 5 에 따르면 Rank 증가에 따라 Test Performance 가 향상되는 경향을 보였으나, Trainable Parameter 증가에 비해 성능 향상 폭은 점차 감소함. 이는 일정 수준 이상의 Rank 증가에서 추가적인 Adaptation Capacity 에 대한 Diminishing Returns 가 나타날 가능성을 보여줌.

또한 동일한 Parameter Budget 에서도 Target Module 을 Q,K,V,O 로 확장하는 것보다 Q,V 에 더 높은 Rank 를 할당하는 Configuration 이 높은 성능을 보임. 따라서 단순히 Trainable Parameter 수를 증가시키는 것보다 제한된 Adaptation Capacity 를 어떤 Module 에 어떻게 배분하는지가 중요함을 확인함.

종합적으로 본 실험에서는 Q,V 에 LoRA 를 집중하여 Rank 를 증가시키는 방식이 Target Module 을 확장하는 방식보다 효과적인 Performance–Parameter Trade-off 를 보임. 다만 Rank 증가에 따른 성능 향상은 점차 포화되는 경향을 보여, 높은 Rank 가 항상 Parameter 증가에 비례하는 성능 향상으로 이어지지는 않음을 확인함.

RQ3. LoRA 의 Task Adaptation 은 어느 Layer 에 집중되며, 선택적 Layer 학습으로 성능을 유지할 수 있는가?

All-layer LoRA 의 Layer-wise Update 분석을 기반으로 High-update Layer 와 Low-update Layer 를 선정하고, 각 Layer 집합에만 LoRA 를 적용하여 선택적 Layer 학습의 영향을 분석함.

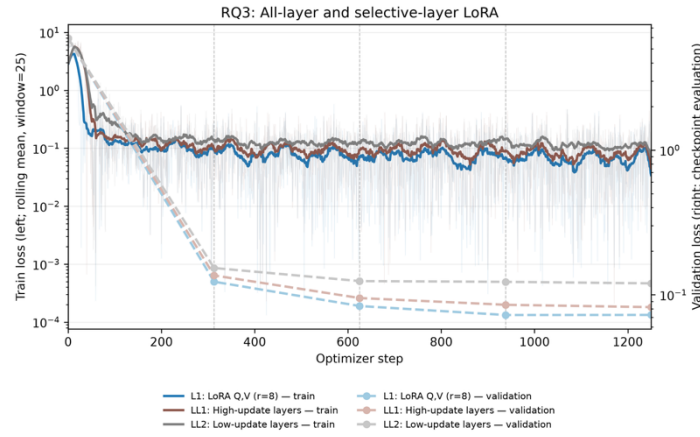


Figure 6. Training and Validation Loss of All-layer and Selective LoRA

Figure 6 에서 보이듯 세 Configuration 모두 학습 과정에서 안정적인 Loss 감소를 보였으나, All-layer LoRA 가 가장 낮은 Validation Loss 를 유지하였으며 High-update Layer 기반 LL1 이 Low-update Layer 기반 LL2 보다 낮은 Validation Loss 를 보임. 이는 동일한 수의 Layer 만 학습하는 조건에서도 선택된 Layer 에 따라 Task Adaptation 정도에 차이가 발생할 수 있음을 보여줌.

Run Adapted Layers	Trainable Params.	Test Acc.	Test Macro-F1	Macro-F1 Retained vs. L1	Training Time	Checkpoint Size
L1 All 28 Layers	1.61M (0.0932%)	0.726	0.509	100.0%	2,488 s	22.3 MB
LL1 High-update: 7, 8, 9, 10	0.23M (0.0133%)	0.698	0.491	96.3%	2,187 s	16.8 MB
LL2 Low-update: 21, 22, 26, 27	0.23M (0.0133%)	0.664	0.466	91.5%	1,927 s	16.8 MB

Table 5. PQA-L Test Performance and Efficiency of All-Layer and Selective Layer LoRA

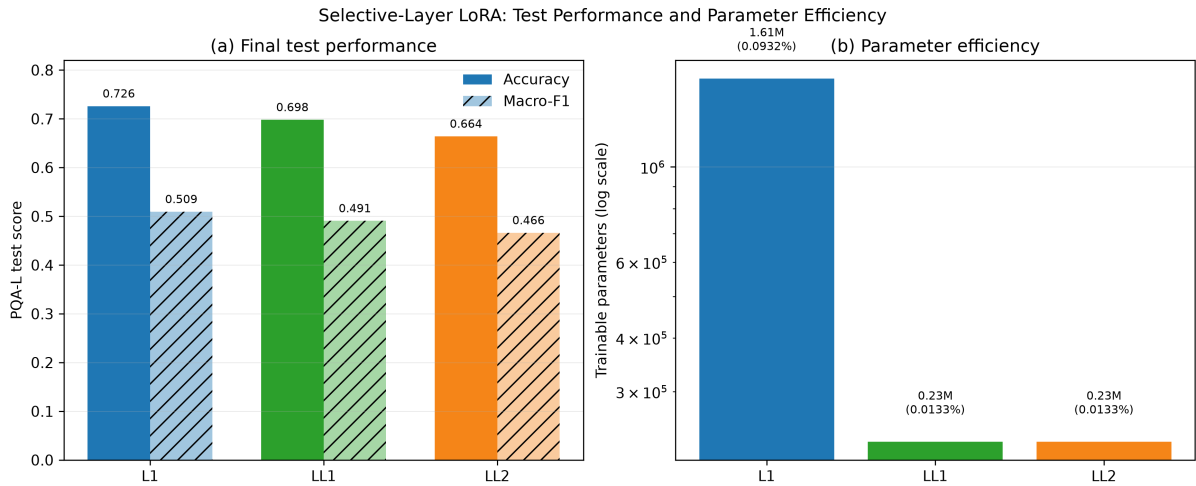


Figure 7. Test Performance and Parameter Efficiency of All-Layer and Selective Layer LoRA

[Table 5](#) 와 [Figure 7](#) 에서 Selective LoRA 는 All-layer LoRA 보다 Trainable Parameters 를 크게 줄이면서도 상당 부분의 Test Performance 를 유지함. 특히 High-update Layer 기반 LL1 은 L1 의 Macro-F1 을 96.3% 유지하였으며, 동일한 Parameter Budget 을 사용하는 Low-update Layer 기반 LL2 보다 높은 성능을 보임. High-update 및 Low-update Layer 의 선정 기준과 Layer 별 Update 분포는 Appendix A6 의 Layer Selection Profile 을 참고.

LL1 과 LL2 는 동일한 수의 Layer 와 Trainable Parameters 를 사용하므로, 두 Configuration 의 성능 차이는 단순한 Parameter 수의 차이보다는 Adapter 가 적용된 Layer 의 위치와 관련된 것으로 해석할 수 있음.

종합적으로, High-update Layer 를 선택적으로 학습하는 방식이 Low-update Layer 를 선택하는 방식보다 효과적인 Task Adaptation 을 보임. 또한 전체 28 개 Layer 가 아닌 일부 Layer 만을 학습하면서도 All-layer LoRA 의 성능을 상당 부분 유지하여, Layer Selection 을 통해 LoRA 의 Parameter Efficiency 를 추가적으로 개선할 가능성을 확인함.

이러한 결과는 Layer-wise Update Magnitude 가 Task Adaptation 과 관련된 Layer 를 식별하는 하나의 기준으로 활용될 가능성을 보여주며, 이를 기반으로 필요한 Layer 에 선택적으로 Adaptation Capacity 를 배분할 수 있음을 시사함. 이는 향후 불필요한 Parameter Update 를 제한함으로써 Overfitting 이나 Catastrophic Forgetting 을 완화하는 Selective Fine-Tuning 전략으로 확장할 수 있음. 다만 본 실험에서는 두 현상을 직접 평가하지 않았으므로, 이에 대한 효과는 추가적인 검증이 필요함.

6. 결론

본 연구에서는 PubMedQA 를 대상으로 Full Fine-Tuning 과 LoRA Fine-Tuning 을 비교하고, Parameter Update 의 범위와 배치가 Task Adaptation 의 성능 및 효율에 미치는 영향을 분석함.

실험 결과, LoRA 는 전체 Parameter 를 Update 하지 않고도 Full Fine-Tuning 과 동등한 수준의 Task Performance 를 확보하면서 Parameter, Training Time 및 Storage 측면에서 높은 Efficiency 를 보임. 이를 통해 해당 Task 에서는 제한된 Parameter Update 만으로도 충분한 Adaptation 이 가능함을 확인함.

LoRA Configuration 분석에서는 Trainable Parameter 를 단순히 증가시키는 것보다 Rank 와 Target Module 을 통해 Adaptation Capacity 를 어떻게 배분하는지가 중요하게 나타남. 특히 Rank 증가에 따른 성능 향상은 점차 제한적으로 나타났으며, 동일한 Parameter Budget 에서도 Target Module 구성에 따라 성능 차이가 발생함.

Layer-wise 분석에서는 LoRA Update 가 모든 Layer 에 균등하게 형성되지 않고 특정 Layer 에 상대적으로 집중되는 경향을 보임. 또한 High-update Layer 를 선택적으로 학습한 경우 Low-update Layer 보다 높은 성능을 보여, Layer-wise Update Magnitude 가 Task Adaptation 과 관련된 Layer 를 식별하는 하나의 기준으로 활용될 가능성을 확인함.

종합적으로 Fine-Tuning 의 효율성은 단순히 학습하는 Parameter 의 수뿐만 아니라, Adaptation Capacity 를 어느 Module 과 Layer 에 배치하는지에 따라 달라질 수 있음을 확인함. 향후에는 이러한 Layer-wise Update 특성이 다른 Task 와 Model 에서도 유지되는지 검증하고, Selective Update 가 Overfitting 이나 Catastrophic Forgetting 완화에도 기여할 수 있는지 추가적인 분석이 필요함.

Appendix

Appendix A. Supplementary Experimental Details

본 Appendix 에서는 본문의 실험을 재현하고 결과를 해석하는 데 필요한 추가적인 Prompt, Training Configuration, Base Model 선정 결과, LoRA Configuration 및 Checkpoint Selection 기준을 제시함.

A1. Prompt Template

모든 Baseline Evaluation 및 Fine-Tuning 실험에서 동일한 Instruction Format 을 사용함. 별도의 Few-shot Example 이나 Chain-of-Thought 를 제공하지 않는 Zero-shot Prompt 로 구성하였으며, 주어진 Question 과 Abstract Context 만을 이용하여 yes, no, maybe 중 하나를 출력하도록 제한함.

System Prompt:

You are a biomedical question-answering assistant. Answer the question based only on the provided abstract context.

User Prompt:

Choose exactly one answer: yes, no, or maybe.

Question:

{question}

Context:

1. {context_1}

...

N. {context_N}

Respond with only one word: yes, no, or maybe.

Assistant:

{answer}

A2. Training Configuration

Full Fine-Tuning 과 LoRA Fine-Tuning 에 적용한 공통 Training Configuration 과 Method-specific 설정을 정리함. 두 방법은 동일한 Dataset, Batch Configuration 및 Optimizer 설정을 사용하고, Learning Rate 와 Gradient Checkpointing 적용 여부를 구분함.

Configuration	Full Fine-Tuning	LoRA Fine-Tuning
Base Model	Qwen3-1.7B	
GPU	RTX 3090 × 1	
Precision	BF16	
Epoch	1	
Per-device Batch Size	1	
Gradient Accumulation	8	
Effective Batch Size	8	
Learning Rate	2×10^{-5}	1×10^{-4}
Optimizer	AdamW	
Weight Decay	0.01	
Warmup Ratio	0.03	
Gradient Clipping	1.0	
Gradient Checkpointing	적용	미적용
Seed	42	

Table A1. Training Configuration for Full Fine-Tuning and LoRA Fine-Tuning

A3. Base Model Selection

Full Fine-Tuning 과 LoRA 비교에 사용할 Base Model 을 결정하기 위해 후보 모델의 Zero-shot Performance 와 Inference Resource Usage 를 사전 평가함. 최종 Base Model 은 성능 순위만을 기준으로 하지 않고, Dense Architecture, Instruction Compatibility 및 단일 RTX 3090 에서 Full Fine-Tuning 이 가능한 모델 규모를 고려하여 Qwen3-1.7B 로 선정함.

Model	Parameters	Accuracy	Macro-F1	Samples/sec	GPU Memory (GB)
Llama-3.2-1B-Instruct	1B	0.116	0.083	42.927	2.853
Llama-3.2-3B-Instruct	3B	0.734	0.511	13.813	7.029
Qwen3-0.6B	0.6B	0.536	0.291	47.124	1.909
Qwen3-1.7B	1.7B	0.614	0.411	24.464	4.151
Qwen3-4B	4B	0.378	0.338	9.905	8.812

Table A2. Zero-shot Performance and Resource Usage of Candidate Base Models

Note. Qwen3-1.7B 는 최고 Zero-shot Performance 를 기준으로 선택한 것이 아니라, 본 실험에서 요구하는 Architecture 와 Hardware Feasibility 를 함께 고려하여 선정함.

A4. LoRA Configuration

RQ2 와 RQ3 에서 사용한 LoRA Configuration 을 정리함. Target Module, Rank 및 Layer Scope 를 변경하여 Configuration 에 따른 Performance 및 Parameter Efficiency 차이를 비교함.

L1, L3, L4 에서는 Rank 변화에 따른 영향을 비교하기 위해 Scaling Ratio $\frac{\alpha}{r} = 2$ 을 동일하게 유지함.

Run	Target Modules	Rank r	Alpha α	Dropout	Bias	Layer Scope
L1	Q, V	8	16	0	None	All
L2	Q, K, V, O	8	16	0	None	All
L3	Q, V	4	8	0	None	All
L4	Q, V	16	32	0	None	All
LL1	Q, V	8	16	0	None	High-update
LL2	Q, V	8	16	0	None	Low-update

Table A3. LoRA Configurations for Rank, Target Module, and Layer Selection Experiments

Note. All 은 모든 Transformer Layer 에 LoRA 를 적용함을 의미하며, High-update 와 Low-update 는 L1 의 Layer-wise Update 분석 결과를 기반으로 선정된 Layer 집합을 의미함.

A5. Checkpoint and Model Selection

모든 Training Run 에서 동일한 Training Progress 를 기준으로 Checkpoint 를 저장하고 Validation 을 수행함. 이를 통해 서로 다른 Fine-Tuning Method 와 LoRA Configuration 의 학습 진행 상태를 동일한 기준에서 비교함.

Training Progress	Optimizer Step	Usage
0%	0	Initial State
25%	313	Validation / Analysis
50%	625	Validation / Analysis
75%	938	Validation / Analysis
100%	1,250	Validation / Analysis

Table A4. Checkpoint Schedule for Validation and Update Analysis

최종 Test Evaluation 에 사용할 Checkpoint 는 다음 우선순위로 선택함.

1. Validation Macro-F1
2. Validation Accuracy
3. Validation Loss

A6. Layer Selection Profile

All-layer LoRA(LL1)의 최종 Checkpoint 에서 측정한 Layer-wise Update Magnitude 를 기준으로 Transformer Layer 를 Ranking 함. 상위 4 개 Layer 를 High-update Layer, 하위 4 개 Layer 를 Low-update Layer 로 선정하여 각각 LL1 과 LL2 의 Selective LoRA Target Layer 로 사용함.

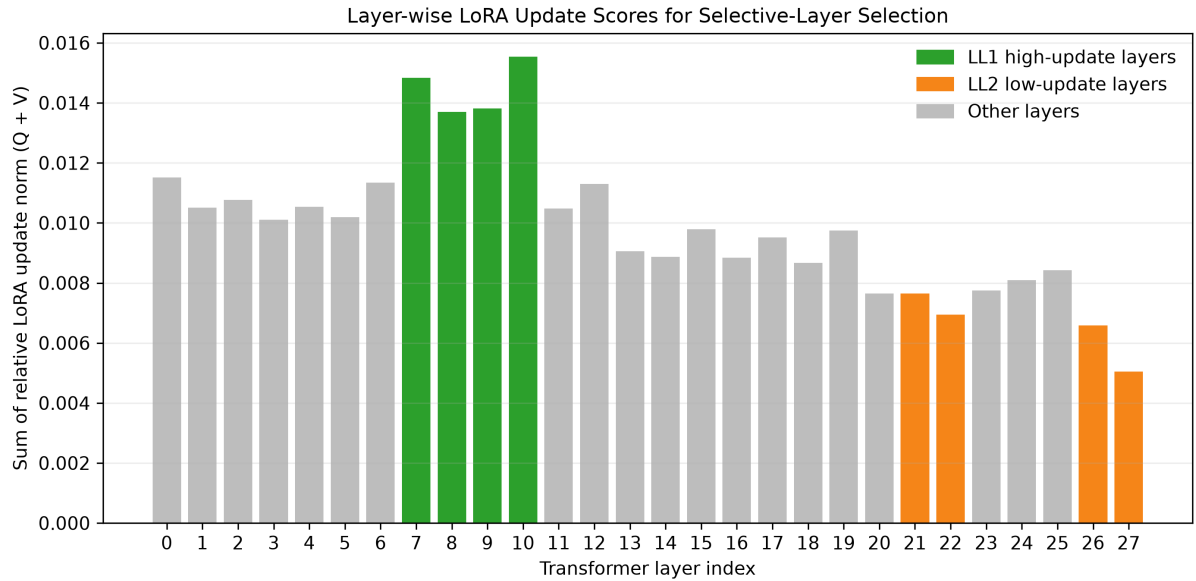


Figure A1. Layer-wise Update Magnitude and Selected Layers for Selective LoRA

High-update 및 Low-update Layer 의 실제 선정 결과를 표시하며, 해당 Layer 를 적용한 LoRA Configuration 은 [Appendix A4](#) 의 LL1 및 LL2 를 참고.

A7. Final Layer-wise Update Heatmap

Full Fine-Tuning 과 L1 의 최종 Checkpoint 에서 Layer 및 Module 별 Relative Update 를 비교함. 이를 Layer × Module 형태의 Heatmap 으로 나타내어, 두 Fine-Tuning 방식에서 Parameter Update 가 어느 Layer 와 Component 에 상대적으로 집중되는지 확인함.

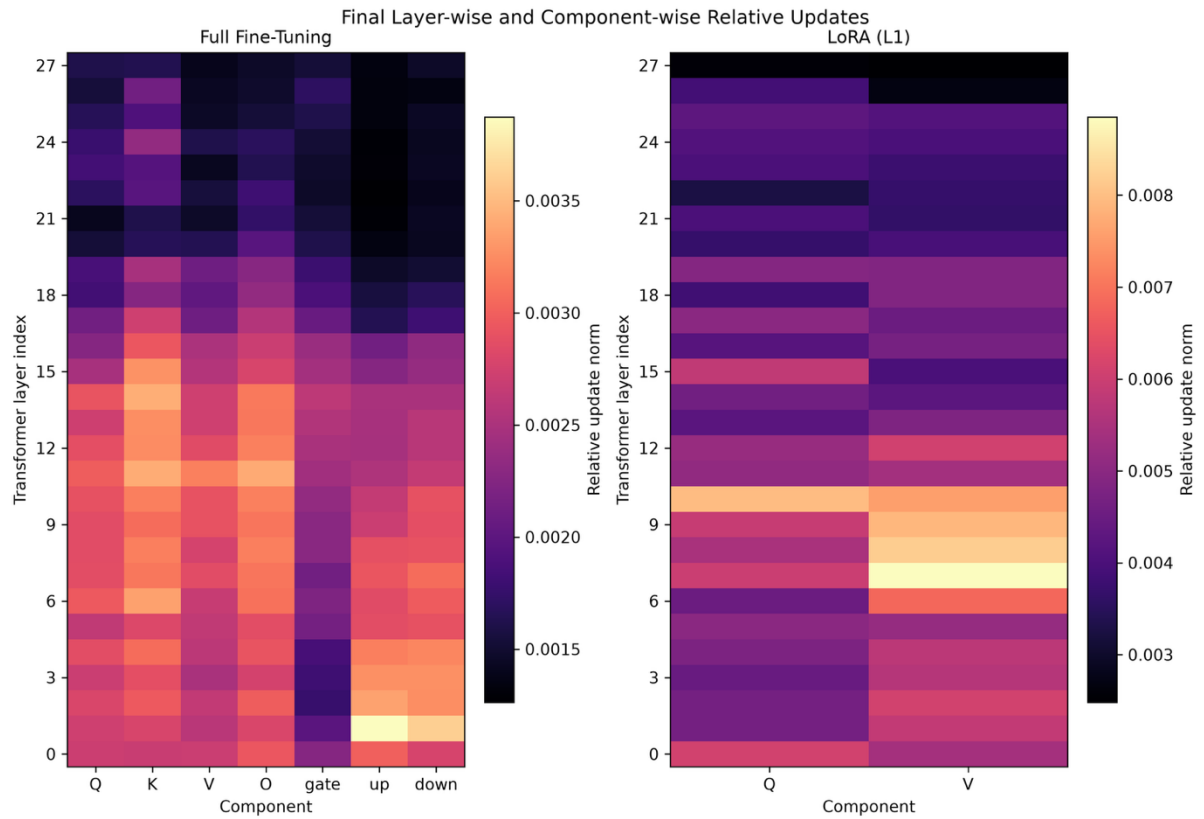


Figure A2. Final Layer-wise Update Magnitude of Full Fine-Tuning and LoRA

Layer Ranking 및 Selective LoRA 의 Target Layer 선정 과정은 [Appendix A6](#) 의 Layer Selection Profile 을 참고.

A8. Temporal Layer-wise Updates

학습 진행에 따른 Layer 별 Update 변화를 비교하기 위해 각 Checkpoint 에서 Layer-wise Update Magnitude 를 측정함. 이를 통해 특정 Layer 의 Update 가 학습 초기에 형성되는지, 후반까지 지속적으로 변화하는지 확인함.

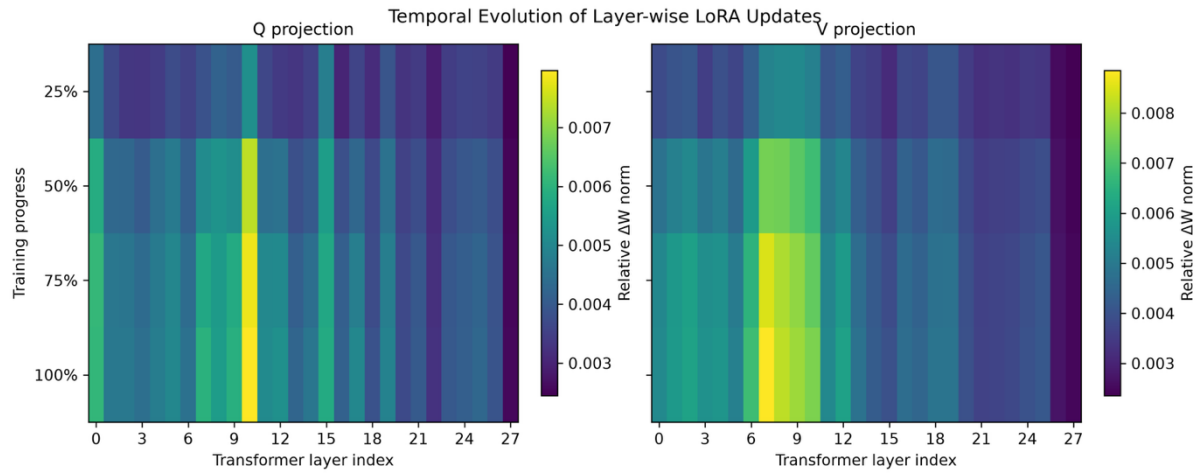


Figure A3. Temporal Evolution of Layer-wise LoRA Updates across Training Checkpoints

각 시점의 Checkpoint 정의 및 Optimizer Step 은 [Appendix A5](#) 를 참고.

A9. LoRA Update Dynamics

각 Training Checkpoint 에서 LoRA Matrix A , B 를 이용하여 Effective Update 를 복원하고, 학습 과정에서 Low-rank Update 의 크기와 변화량이 어떻게 형성되는지 분석함.

이를 통해 LoRA Update 가 학습 초기에 빠르게 형성되는지, 이후에도 지속적으로 변화하는지 확인함

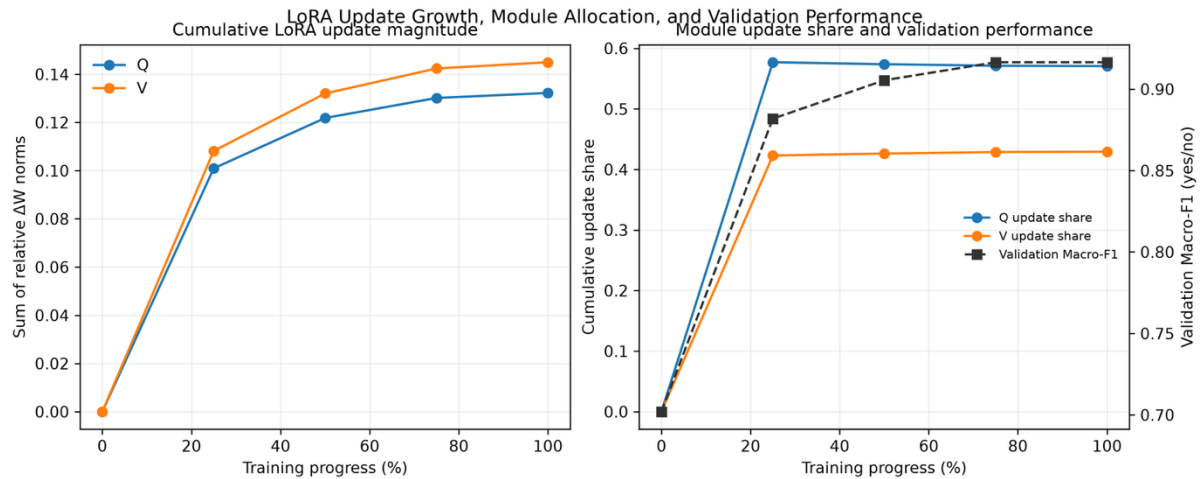


Figure A4. LoRA Update Dynamics across Training Progress

분석에 사용한 Checkpoint 시점은 [Appendix A5](#), Rank 및 Scaling Factor 설정은 [Appendix A4](#) 를 참고.

A10. Effective Rank Trajectory

LoRA 의 Configured Rank 가 실제 Update 에서 어느 정도 활용되는지 확인하기 위해 각 Checkpoint 의 Effective Update 를 기반으로 Effective Rank 의 시간적 변화를 분석함.

이를 통해 학습 초기에 제한된 Low-rank Direction 만 사용하다가 학습이 진행되면서 더 많은 Direction 을 활용하는지, 또는 일정 수준에서 포화되는지 확인함.

각 LoRA Update Matrix ΔW 의 Effective Rank 는 Singular Value Spectrum 을 기반으로 한 Entropy-based Effective Rank 로 계산함. ΔW 의 Singular Value 를 σ_i 라 할 때,

$$p_i = \frac{\sigma_i^2}{\sum_j \sigma_j^2}, \quad \text{erank} \Delta W = \exp - i \sum p_i \log p_i$$

로 정의함. 모든 Singular Value 가 0 인 경우 Effective Rank 는 0 으로 정의함. 이는 Threshold 기반의 Numerical Rank 가 아니라, LoRA Update 의 Singular Value 가 여러 Direction 에 얼마나 분산되어 활용되는지를 나타내는 Soft Rank 지표임. Figure A5 에서는 각 Checkpoint 에서 Q/V Adapter 별 Effective Rank 를 계산한 후 전체 Adapter 의 평균과 표준편차를 나타냄.

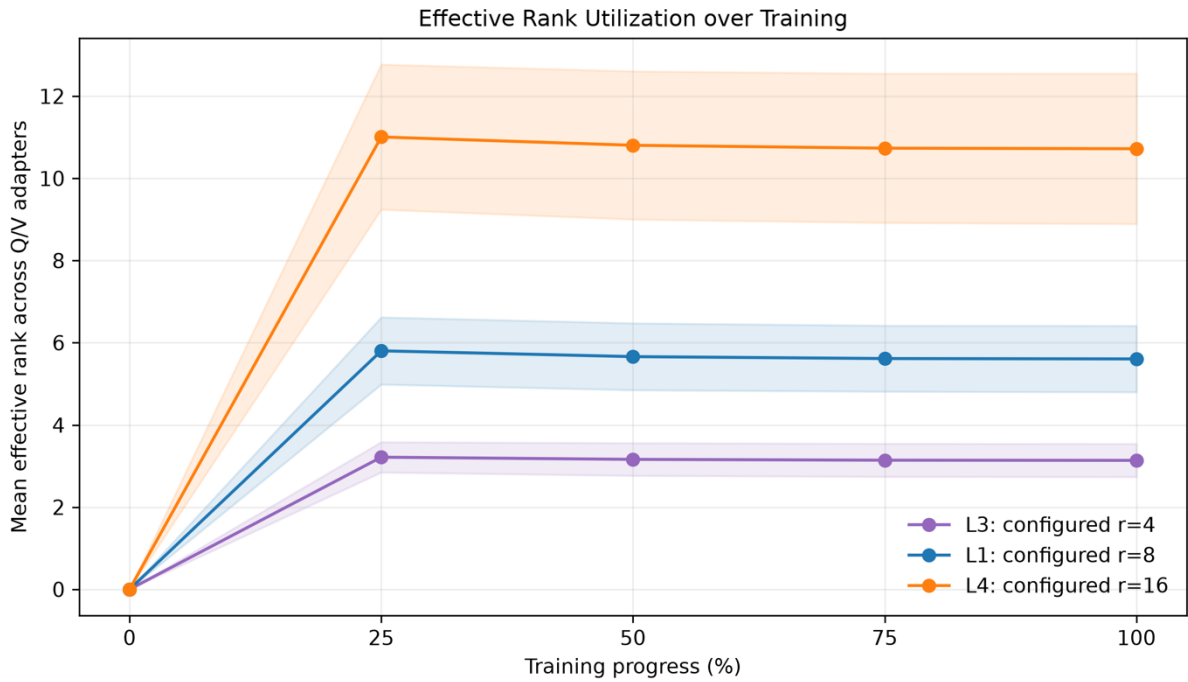


Figure A5. Effective Rank Trajectory across LoRA Rank Configurations

L3, L1, L4 의 Configured Rank 는 각각 4,8,16이며, 상세 Configuration 은 [Appendix A4](#) 를 참고.

Effective Rank 계산의 기반이 되는 Singular Value 분포는 [Appendix A11](#) 에 제시함.

A11. Singular Value Spectra

LoRA Effective Update 에 Singular Value Decomposition(SVD)을 적용하여 Low-rank Update 가 각 Rank Dimension 을 어떻게 활용하는지 분석함.

$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r)$$

Rank Configuration 별 Singular Value Spectrum 을 비교하여 Update 가 소수의 Dominant Direction 에 집중되는지 또는 여러 Direction 에 분산되는지 확인함.

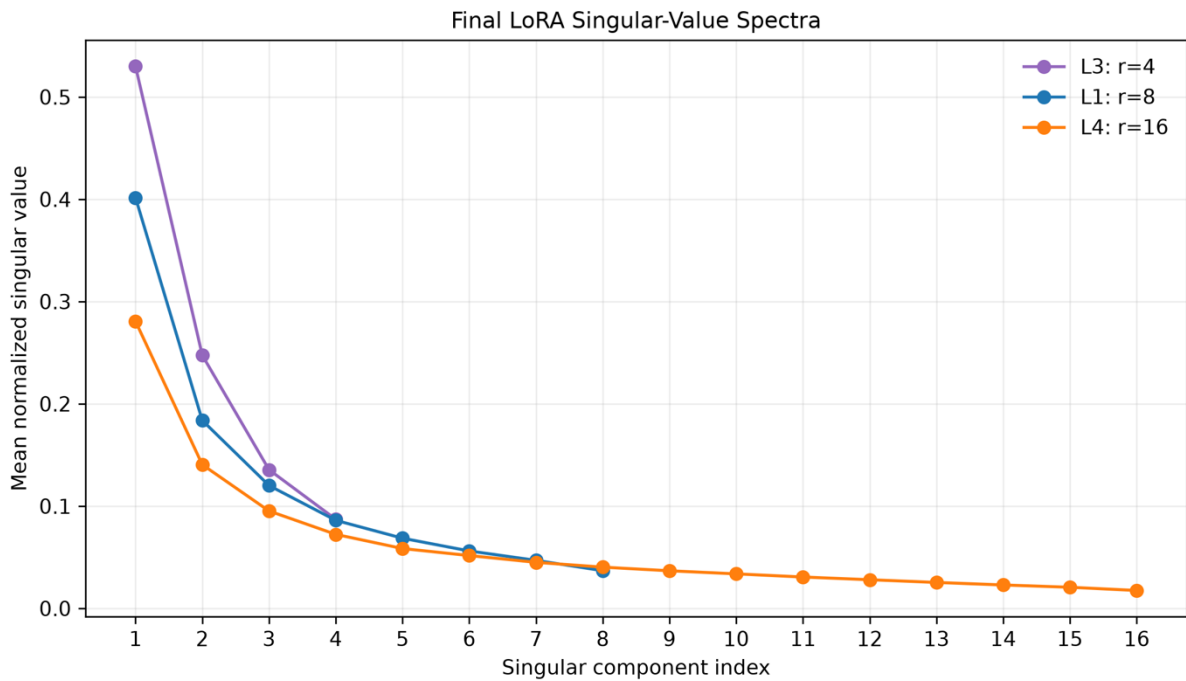


Figure A6. Singular Value Spectra of LoRA Updates across Rank Configurations

비교 대상인 L3($r = 4$), L1($r = 8$), L4($r = 16$)의 상세 Configuration 은 [Appendix A4](#) 를 참고. Singular Value 를 기반으로 계산한 시간별 Effective Rank 결과는 [Appendix A10](#) 에 제시함.